FC7xxx SSI Integration Manual

Rev.0.1

Revision History

Revision	Date	Changes
0.1	2023/12/15	Initial release

Table of Contents

Chapter 1 Introduction

1.1 Introduction

This integration manual describes the integration requirements for the SSI module.

Chapter 2 Building

2.1 Dependencies on Other Modules

Module configuration dependency

- Mcu: This module provides the clock reference point for SSI module
- Port: This module provides the port configurations if want output SSI signal through pin.
- Common: This module is the basic module which used to choose the chip.
- OS: This is used for SSI main function reference.
- EcuM: is required for selecting the reference to the wakeup source for every SSI controller.
- Det: is required for signaling the default error detection (parameters out of range, null pointers, etc)

Module build dependency

- Common: This module provides common type for other modules.
- Rte: This module provides APIs to protect/unprotect some parts of code from interrupts (Exclusive Areas).
- Det: This module is necessary for tracing Development errors.
- EcuM: This module provides the EcuM callback functions when wakeup support is enabled.

Module initialization dependency

- Mcu: To use the SSI module, the user needs to initialize the MCU module first.
- Port: To output SSI signal, the user needs to initialize PORT module first and assign the UART pins.

2.2 Files Required for Compile

SSI module files:

- _MCAL/Src/Ssi/Src/CDD_Ssi.c
- _MCAL/Src/Ssi/Src/Ssi_Hal.c
- _MCAL/Src/Ssi/Src/Ssi_Isr.c
- _MCAL/Src/Ssi/include/CDD_Ssi.h
- _MCAL/Src/Ssi/include/Ssi_Hal.h
- _MCAL/Src/Ssi/include/Ssi_HWA.h
- _MCAL/Src/Ssi/include/Ssi_Reg.h
- _MCAL/Src\Base/include/MemMap/Ssi_MemMap.h

- _MCAL_generate/src/CDD_Ssi_cfg.c
- _MCAL_generate/include CDD_Ssi_cfg.h

Common module files:

- _MCAL/Src/Common/include/Mcal.h
- _MCAL/Src/Common/include/Std_Types.h
- _MCAL/Src/Common/include/Platform_Types.h
- _MCAL/Src/Common/include/Compiler.h
- _MCAL/Src/Common/include/Compiler_Cfg.h
- _MCAL/Src/Common/include/CompilerDefinition.h

Det module files:

- Det.h

Rte module files:

- SchM_Ssi.h

2.3 Add Plug-ins

SSI module plug-ins are developed for EB tresos Studio, so, to use SSI plug-ins on the EB tresos Studio, the user needs to add the SSI module to the EB plug-ins folder first.

1) Copy the SSI module (_MCAL/EB_Plugins/eclipse/plugins/Ssi) folder to EB tresos plug-ins (EB/tresos/plugins/) folder.
2) Set the SSI module output location folder for the generated source file and header files.
3) Use EB tresos to configure the SSI module and generate source and header files.

Chapter 3 Memory

3.1 Sections in Memory Map

Section Name	Section Type	Description
SSI_START_SEC_CONFIG_DATA_8 SSI_STOP_SEC_CONFIG_DATA_8 SSI_START_SEC_CONFIG_DATA_16 SSI_STOP_SEC_CONFIG_DATA_16 SSI_START_SEC_CONFIG_DATA_32 SSI_STOP_SEC_CONFIG_DATA_32	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by startup code(data).
SSI_START_SEC_CONFIG_DATA_UNSPECIFIED SSI_STOP_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	Start and stop of Memory Section for Config Data.
SSI_START_SEC_CONST_BOOLEAN SSI_STOP_SEC_CONST_BOOLEAN SSI_START_SEC_CONST_8 SSI_STOP_SEC_CONST_8 SSI_START_SEC_CONST_16 SSI_STOP_SEC_CONST_16 SSI_START_SEC_CONST_32 SSI_STOP_SEC_CONST_32 SSI_START_SEC_CONST_ UNSPECIFIED SSI_STOP_SEC_CONST_ UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit or boolean. These variables are read only (rodata).
SSI_START_SEC_CODE SSI_STOP_SEC_CODE SSI_START_SEC_ RAMCODE SSI_STOP_SEC_ RAMCODE SSI_START_SEC_CODE_AC SSI_STOP_SEC_CODE_AC	Code	Start and stop of memory Section for Code (text).
SSI_START_SEC_VAR_NO_INIT_BOOLEAN SSI_STOP_SEC_VAR_NO_INIT_BOOLEAN SSI_START_SEC_VAR_NO_INIT_8 SSI_STOP_SEC_VAR_NO_INIT_8 SSI_START_SEC_VAR_NO_INIT_16 SSI_STOP_SEC_VAR_NO_INIT_16 SSI_START_SEC_VAR_NO_INIT_32 SSI_STOP_SEC_VAR_NO_INIT_32 SSI_START_SEC_VAR_NO_INIT_ UNSPECIFIED SSI_STOP_SEC_VAR_NO_INIT_ UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are never cleared and never initialized by startup code (bss).
SSI_START_SEC_VAR_ INIT_BOOLEAN SSI_STOP_SEC_VAR_ INIT_BOOLEAN SSI_START_SEC_VAR_ INIT_8 SSI_STOP_SEC_VAR_ INIT_8 SSI_START_SEC_VAR _INIT_16 SSI_STOP_SEC_VAR _INIT_16 SSI_START_SEC_VAR _INIT_32 SSI_STOP_SEC_VAR _INIT_32	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by startup code (data).

Section Name	Section Type	Description
SSI_START_SEC_VAR _INIT_ UNSPECIFIED SSI_STOP_SEC_VAR _INIT_ UNSPECIFIED		

Chapter 4 Exclusive Area

SSI module using the services of Schedule Manger (SchM) for entering and exiting critical regions.

The following critical regions are used in the SSI driver:

- SSI_Hal.c:
  - SchM_Enter_Ssi_SSI_EXCLUSIVE_AREA_00 is used in Ssi_HL_Instance_Init.
  - SchM_Enter_Ssi_SSI_EXCLUSIVE_AREA_01 is used in Ssi_HL_Instance_Init.
  - SchM_Enter_Ssi_SSI_EXCLUSIVE_AREA_02 is used in. Ssi_LL_Subinstance_Init
  - SchM_Enter_Ssi_SSI_EXCLUSIVE_AREA_05 is used in Ssi_HL_Instance_DeInit.
  - SchM_Enter_Ssi_SSI_EXCLUSIVE_AREA_06 is used in Ssi_LL_Subinstance_DeInit.
  - SchM_Enter_Ssi_SSI_EXCLUSIVE_AREA_07 is used in Ssi_HL_GetMessage.
  - SchM_Enter_Ssi_SSI_EXCLUSIVE_AREA_08 is used in Ssi_HL_GetMessage.

Chapter 5 Interrupt Service Routine (ISR)

Instance	Interrupt Name	IRQ Number (NVIC Interrupt ID)
SSI0	SSI_IsrSSI0_All	155

Chapter 6 Error Report

6.1 Det

Function Name	Error Type
Ssi_Init	SSI_INITIALIZED; SSI_E_INIT_FAILED_U8; SSI_E_PARTITION_MAPPING; SSI_E_PARAM_U8; SSI_E_TIMEOUT_U8;
Ssi_GetVersionInfo	SSI_E_PARAM_U8;
Ssi_DeInit	SSI_E_PARTITION_MAPPING ; SSI_E_UNINIT_U8;
Ssi_MainFunctionMessageRead	SSI_E_UNINIT_U8;

6.2 Dem

None

Chapter 7 Function Calls

7.1 Function Calls during Startup

SSI shall be initialized during STARTUP phase of EcuM initialization. The API to be called for this is SSI_Init(). The MCU module should be initialized before the SSI is initialized. The SSI module shall be initialized by SSI_Init(<&SSI_Configuration>) service call during the start-up before the SSI peripherals are used. Please note that GPIO pins used for connection of SSI physical layer have to be properly assigned to the desired module prior to the SSI initialization: so the MCU and PORT modules shall be initialized before SSI is initialized. After the SSI module is initialized, each SSI controller has to be initialized as well before using it. This is also done by the SSI_Init(<&SSI_Configuration>) service.

7.2 Function Calls during Shutdown

None

7.3 Function Calls during Wake-up

None

7.4 Function Calls during Runtime

Users should call Ssi_MainFunctionMessageRead in period to get the Ssi message to avoid to lost the message. They should be call while MsgReadType is configurated as Polling in EB Tresos.

Chapter 8 Other Requirements

8.1 Notification, Callback, Callout

- Notification
  There are no callback functions within the SSI driver.
- Callback
  There are no callback functions within the SSI driver.
- Callout
  The SSI Driver uses these mandatory callouts functions which have to be provided by the respective SSI stack: SsiNotification.

8.2 Macros

Please check various definitions available in Common module's include file Mcal.h for details.

- If OS is not used:
  AUTOSAR_OS_NOT_USED needs to be defined.
- If OS is used:
  Do not define AUTOSAR_OS_NOT_USED.

Chapter 9 Integration Steps

1) Configure the SSI module and generate configuration files (please refer to Building chapter for details).
2) Configure appropriate memory sections in linker file or other (please refer to Memory chapter for details).
3) Map interrupt notification to their vector locations (please refer to chapter ISR for details).
4) Build the SSI module with dependent modules.